

Polynomial sparse incidence and parity profiles from certified interval orbits

A proof-carrying local improvement for the
ProveIt unknot-recognition program

Research continuation prepared for the ProveIt project
Prepared with ChatGPT; mathematical and software review invited

8 October 2026

Abstract

The maintained interval-incidence routine in ProveIt computes a dense table with one entry for each subset of the supplied ports. Even exact sharing of repeated marked unions leaves an unavoidable 2^n output cost. We replace that output contract by the positive incidence signatures and their multiplicities. A consequence of the weighted Agol–Hass–Thurston theorem is that this sparse output has polynomial size in the binary interval description, including the number of ports; logarithmically many ports are not required for this local problem.

We give a self-contained nonnegative-measure recovery argument, implement it using only the existing unweighted orbit counter, and provide an independent certificate checker. Recovery uses $O(sr)$ subset-sum requests for s positive signatures. Balanced block deletion improves this to an output-cardinality-sensitive bound. A certificate retains only atom weights and zero witnesses for retained ports, not the complete exploratory trace. For signed relations, a parity double has exactly the same signature support, allowing per-signature consistency counts with at most $s + 1$ further orbit counts.

The delivered Python modules are tested against immutable, Git-blob-checked native source. Exact graph comparisons, exhaustive small-support checks, adversarial replay, large binary inputs, paired benchmarks, and negative controls are included. This is an asymptotic improvement over the repository’s dense incidence implementation, not a new general theorem about weighted orbit counting and not a quasi-polynomial unknot recognizer. Attachment maps, geometric provenance, and the global hierarchy bounds remain separate obligations.

Status. Research draft with complete proofs relative to the stated classical orbit theorems, executable tests, and independently replayed certificates. Not peer-reviewed or proof-assistant verified. Ordinary knot-recognition dispatch is unchanged.

Contents

1	Objective, repository baseline, and contribution	3
1.1	Three levels that must not be conflated	3
1.2	Relationship to established results	3
1.3	Main result	4
2	Interval relations, signatures, and polynomial support	4
2.1	Input and output conventions	4
2.2	The classical compressed-output fact	5
3	Certified subset sums by coning marked unions	6
4	Sparse recovery by positive residuals	6
4.1	One-pass minimal positive support	7
4.2	Balanced block deletion	8
4.3	What the query bound does not say	8
5	Certificates without the exploratory search trace	9
5.1	Why positivity is an essential hypothesis	10
5.2	Independent implementation and trust boundary	10
6	Per-signature parity profiles without a second support search	11
7	Bit complexity and the precise quasi-polynomial consequence	12
7.1	Input size and oracle-sensitive accounting	12
7.2	Closing the local complexity argument	13
7.3	A conditional transfer statement for a hierarchy	13
8	Validation and controlled performance measurements	13
8.1	Correctness checks actually performed	14
8.2	Main benchmark design	14
8.3	Longer-batch follow-up and timing uncertainty	15
8.4	Large port sets and a geometric negative control	16
8.5	Parity support reuse	17
9	Software contract and integration	17
9.1	Additive modules rather than a hidden dispatch change	17
9.2	Resource accounting	18
9.3	Minimal reproduction	18
10	Further research questions and concrete proof targets	18
10.1	A direct, certified weighted orbit implementation	18
10.2	Explicit support bounds in breakpoint parameters	19
10.3	Better coning plans for separated marks	19
10.4	Adaptive recovery and certified residual data structures	19
10.5	Stable port families under local updates	19
10.6	From incidence sets to ordered attachment data	19
10.7	Finite covers and topology by signature	20
10.8	The global size and completeness theorem	20
11	Conclusion	20
A	A completely explicit 17-point example	20
B	Certificate transport and source provenance	21
C	Reproducibility and claim ledger	21

1 Objective, repository baseline, and contribution

The long-term objective is a recognizer whose worst-case bit complexity on an explicit n -crossing diagram is $n^{O(\log n)}$. The present contribution addresses a concrete interface in the compressed-topology part of that program: counting components according to the supplied boundary or attachment regions that they meet. This is useful information, but it is not the full combinatorial description of an attachment.

The inspected maintained files are `fast/fastunknot/interval_orbits.py`, `interval_orbit_verify.py`, and `interval_incidence.py`, together with `synthesis/orbit_certificates.tex`. The last of these explicitly distinguishes incidence from order, slopes, and future attachment maps. Its current algorithm makes coned-union orbit queries, shares queries whose normalized marked unions agree, and applies a Boolean Möbius transform to a dense array. With r ports, the array still has 2^r slots. Its sufficient local parameter regimes therefore restrict r to logarithmic or squared-logarithmic size relative to an external input parameter [5].

This restriction is an artifact of the output format, not a necessary restriction for the sparse incidence problem. The weighted orbit theorem already implies a polynomial-size compressed answer. The implementation supplied here obtains that answer without adding a weighted orbit scheduler: it reuses the current unweighted scheduler and its independent local proof system. This reduces the amount of new trusted algorithmic machinery.

1.1 Three levels that must not be conflated

An *orbit assertion* concerns an explicitly supplied system of interval identifications. A *topological assertion* additionally requires a correct construction of that interval system from a surface, together with whatever labels are being interpreted geometrically. A *knot-recognition assertion* requires valid provenance from the source knot and a complete recognition argument. The delivered modules establish the first level. They can participate in the second when given a validated adapter. They do not independently establish the third.

Lackenby’s 2021 announcement motivates the quasi-polynomial target [4]. His July 2026 preprint develops incompressible-surface and hierarchy algorithms, while its running-time discussion leaves important bounds unspecified [3, Section 9]. We neither replace that program nor infer its missing global bounds from a faster incidence routine.

1.2 Relationship to established results

Agol, Hass, and Thurston (AHT) prove polynomial-time unweighted and weighted orbit algorithms for binary interval-pairing systems [1]. Their weighted output is compressed: many orbits of equal weight can share one record. Chakrabarty and Huang already give $O(rs)$ -query reconstruction of a coverage function with s positive support sets [2, Theorem 2.2]. We do not claim either fact as a new discovery. Our recovery proof uses minimal positive residual sets rather than reproducing their partition-refinement implementation.

The concrete contribution is the composition needed by this repository: polynomial sparse incidence over the existing unweighted kernel; a compact, source-bound minimality certificate; an independently implemented checker; balanced deletion as an optional query policy; support reuse for parity profiles; and a measured, additive integration package. The literature search supports these attributions, not a claim of exhaustive priority verification for every refinement in this report.

1.3 Main result

Theorem 1.1 (Polynomial sparse incidence, with replay). *Let a finite integer interval of size N carry k interval isometries, and let r explicitly listed ports be unions of altogether M marked intervals. Put $B = \lceil \log_2(N + 2) \rceil$. The nonzero component-incidence histogram can be computed in*

$$\text{poly}(k + M + r + B)$$

bit time and space. It admits a certificate of polynomial bit length, checked in polynomial time without running an orbit-count producer or enumerating the 2^r subsets. No a priori sparsity estimate is required by the algorithm.

For an arbitrary nonnegative incidence measure with s positive signatures, the recovery procedure uses $O(sr + 1)$ subset-sum requests. In the interval setting, polynomial support size follows from the weighted AHT theorem. The delivered implementation realizes the recovery and certificate layers over the maintained unweighted kernel.

The theorem is proved by combining Lemma 2.2, Theorem 4.2, and Theorem 5.1; Section 7 gives the bit-complexity accounting. It is a local theorem. In particular, its polynomial is measured in the *current encoded interval input*, not in the crossing number of an earlier knot diagram whose relationship to that input has not been bounded.

2 Interval relations, signatures, and polynomial support

2.1 Input and output conventions

Let $X = \{0, \dots, N - 1\}$. A pairing identifies each point of an inclusive interval $[a, b]$ with its image in an equal-length inclusive interval $[c, d]$, by a translation or reflection. Pairing endpoints are binary integers. The generated equivalence relation has orbit set $\mathcal{O}$ and count $C = |\mathcal{O}|$. None of these definitions requires expansion into individual points.

A port $A_i \subseteq X$ is a finite union of half-open integer intervals $[u, v)$. Empty intervals, empty ports, overlaps, coincidences, and adjacent intervals are allowed. Normalization deletes empty intervals and merges overlap and adjacency. The difference between inclusive pairing intervals and half-open marked intervals is deliberate; the coning construction below specifies the conversion explicitly.

Definition 2.1 (Incidence histogram). For an orbit O , its signature is

$$T(O) = \{i \in [r] : O \cap A_i \neq \emptyset\}.$$

Define

$$h(T) = |\{O \in \mathcal{O} : T(O) = T\}|, \quad s = |\text{supp } h|.$$

The sparse answer lists only the pairs $(T, h(T))$ for which $h(T) > 0$. In particular, $h(\emptyset)$ counts completely unmarked orbits and is not discarded.

A signature is represented by an r -bit mask. Its coefficient requires at most B bits, since it is bounded by N . The sparse output has $O(s(r + B))$ bits, whereas a literal dense histogram has 2^r entries even when all but one are zero.

Define the subset-sum function

$$G(U) = \sum_{T \subseteq U} h(T), \quad U \subseteq [r]. \tag{1}$$

Thus $G([r]) = C$ and $G(\emptyset) = h(\emptyset)$. It is the total mass of components avoiding every port outside U . Nonnegativity, rather than a cancellation-prone inclusion–exclusion formula, will drive the reconstruction.

2.2 The classical compressed-output fact

We use the following consequence of the weighted AHT theorem [1, Theorem 16]. Given binary interval pairings and a vector weight that is constant on explicitly specified subintervals, the algorithm outputs polynomially many records encoding orbit weights and their multiplicities, in time polynomial in the pairing count, weight dimension, number of constant intervals, and relevant integer bit lengths. The representation preceding that theorem is essential: an output interval can stand for many different orbits with the same vector weight. It is not a promise to write one record for each of exponentially many components.

Lemma 2.2 (Polynomial signature support). *There is a fixed polynomial W such that every interval instance above satisfies*

$$s \leq W(k + M + r + B + 1).$$

For a static relation, $k = 0$, the sharper elementary bound $s \leq 2M + 1$ holds when $N > 0$.

Proof. The cases $N = 0$ and $r = 0$ are immediate. Otherwise give each point the nonnegative vector

$$z(x) = (\mathbf{1}_{A_1}(x), \dots, \mathbf{1}_{A_r}(x)) \in \mathbb{N}^r.$$

Its value changes only at marked endpoints. Consequently, it is constant on at most $2M + 1$ nonempty intervals. These intervals and their vectors can be formed from the supplied endpoints using polynomial work.

For an orbit O , the corresponding weight is

$$z(O) = \sum_{x \in O} z(x).$$

Its i th coordinate is positive exactly when $i \in T(O)$. Coordinate sizes are at most N . A safe bound for the initial total weight parameter is $2 + Nr$, whose bit length is $O(B + \log(r + 1))$; padding zero parameters by one handles degenerate inputs.

Apply the compressed weighted-output theorem. Replace each output weight vector by its coordinatewise positivity mask, and add the multiplicities of records with equal masks. A deterministic polynomial-time computation cannot emit more than polynomially many explicit records. This bounds the number of distinct masks by a fixed polynomial in the displayed parameters. We do not need to know its degree to run the sparse recovery algorithm.

When $k = 0$, every orbit is a point. Its signature is constant on each of the at most $2M + 1$ intervals cut out by marked endpoints, proving the sharper bound directly. $\square$

The lemma concerns explicitly described interval systems. An arbitrary hidden set system can have all 2^r positive signatures; encoding such a system as static point ports may itself require exponentially many intervals. There is no contradiction between the lemma and general coverage-oracle lower bounds.

Remark 2.3 (Why use an unweighted reduction?). The weighted theorem already gives a polynomial algorithm for this mathematical output. The present work does not improve that published classification. Its purpose is to obtain a polynomial sparse interface using the repository’s already implemented, independently replayable unweighted primitives. No weighted-AHT implementation or speed comparison against one is included.

3 Certified subset sums by coning marked unions

Let A be a nonempty normalized union of marked intervals. Adjoin identifications that make all points of A equivalent. For each constituent $[a, b]$ with $b - a \geq 2$, add the translation

$$[a, b - 2] \longrightarrow [a + 1, b - 1]. \quad (2)$$

Choose the left endpoint of the first constituent as a hub and join it to the left endpoint of each other constituent by a singleton pairing. If A has m_A constituents, this adds at most $2m_A$ pairings. It never lists the points of a long interval.

Lemma 3.1 (Coning identity). *Let C_A be the orbit count after this construction and let $H(A)$ be the number of original orbits meeting A . Then*

$$H(A) = C - C_A + 1.$$

Therefore, with $A(U) = \bigcup_{i \notin U} A_i$,

$$G(U) = \begin{cases} C, & A(U) = \emptyset, \\ C_{A(U)} - 1, & A(U) \neq \emptyset. \end{cases} \quad (3)$$

Proof. Each interval translation connects its entire marked interval, and the singleton joins connect those intervals to one another. Exactly the original orbits that meet A become one class. An original orbit disjoint from A meets no new relation and remains unchanged. Thus $C_A = C - H(A) + 1$. A component contributes to $G(U)$ precisely when it avoids $A(U)$, so $G(U) = C - H(A(U))$. The empty-union case must be treated separately: it introduces no relations and no subtraction of one. $\square$

The baseline orbit count is computed once. Different masks with the same normalized complement union share a count and, when requested, a proof. Equality of counts is *not* used as evidence of equality of marked unions. The cache key is the complete canonical union. This preserves the maintained interface’s exact sharing rule while changing which masks are requested.

For certification, the caller supplies the original universe, pairing list, and ports. The count proof must bind to that original pairing list, followed by the independently reconstructed coning pairings in the specified order. An otherwise valid proof for a different union is not accepted. This binding is indispensable: the same numerical count may hold for unrelated marked sets.

4 Sparse recovery by positive residuals

This section temporarily assumes only a nonnegative integer measure h on subsets of $[r]$, access to its exact subset sums G , and its total mass C . The interval construction supplies these hypotheses, rather than attempting to infer them from a few arbitrary oracle values.

After recovering distinct atoms $(T_1, w_1), \dots, (T_j, w_j)$, define

$$R_j(U) = G(U) - \sum_{\ell \leq j, T_\ell \subseteq U} w_\ell. \quad (4)$$

The invariant is that these are the subset sums of the remaining nonnegative measure. In particular, R_j is monotone under inclusion, and a zero value excludes every remaining atom contained in its argument.

4.1 One-pass minimal positive support

Suppose the residual total is positive. Start with $U = [r]$. Examine the ports once, in a fixed order. On examining i , replace U by $U \setminus \{i\}$ when $R_j(U \setminus \{i\}) > 0$; otherwise retain i and record the zero argument. At the end put $T = U$ and $w = R_j(T)$.

Lemma 4.1 (Minimality survives later deletions). *At the end of the pass, $R_j(T) > 0$ and $R_j(T \setminus \{i\}) = 0$ for every $i \in T$. Consequently, the remaining measure has exactly weight $R_j(T)$ at T and no positive atom properly contained in T .*

Proof. A deletion is made only when its resulting residual is positive, so positivity persists. If i is retained, its tested set Z_i has $R_j(Z_i) = 0$. Later steps only delete other elements, whence $T \setminus \{i\} \subseteq Z_i$. Monotonicity and nonnegativity imply $R_j(T \setminus \{i\}) = 0$.

Every proper subset $S \subsetneq T$ omits at least one $i \in T$ and is contained in $T \setminus \{i\}$. Its remaining coefficient must be zero. Therefore the only possible remaining contribution to $R_j(T)$ is the coefficient of T itself. $\square$

The implementation first queries $G(\emptyset)$ and emits the empty atom if positive. This avoids repeatedly searching for it and makes the treatment of unmarked components explicit. It then applies the preceding pass while extracted mass is less than C . It subtracts the *entire* coefficient w , not one component at a time.

Theorem 4.2 (Exact positive-atom peeling). *The procedure recovers all s positive coefficients exactly, with no prior bound on s and with $O(sr + 1)$ subset-sum requests. Its output order extends inclusion on the positive support: every proper positive subset of an emitted set has already been emitted.*

Proof. The first empty coefficient is exact by definition. Assume the earlier coefficients are exact. Equation (4) is then the subset-sum function of the actual remaining nonnegative measure. Lemma 4.1 identifies a positive coefficient exactly, and subtracting it preserves the invariant.

Each iteration removes one distinct support set, so there are at most s iterations. Once the removed mass is C , no nonnegative residual mass remains anywhere. Conversely, before that time the positive total permits another pass. There are at most r deletion trials per nonempty emitted atom, one weight request per atom, and the initial empty request. Raw-value caching can only reduce the number of oracle calls.

If a proper positive subset had not already been removed, the current residual would not be minimal at T . This proves the assertion about the output order. $\square$

Large component multiplicities do not increase the number of iterations. For example, the same number of signatures is recovered whether their coefficients are one or 2^{16000} . Arithmetic costs still

depend on coefficient bit lengths. The raw cache stores $G(U)$, *not* $R_j(U)$, since a residual changes after every extraction.

4.2 Balanced block deletion

The optional **split** strategy replaces singleton trials by a balanced binary tree on the port indices. At a node with index block D , test $R_j(U \setminus D)$. If positive, delete the entire block and do not inspect its children. If zero and D is a singleton, retain its port and record the zero witness. Otherwise split D into its two nearly equal contiguous halves and inspect them successively. There is no branching search over different candidate solutions: both child blocks are processed against the same evolving set U .

Proposition 4.3 (Cardinality-sensitive deletion bound). *The block procedure also emits a minimal positive residual set. If its final size is $d \geq 1$, the number of deletion trials is*

$$O\left(1 + d \log\left(1 + \frac{r}{d}\right)\right),$$

and is at most $2r - 1$. Over the full reconstruction, the trial count is bounded by the sum of this expression over positive nonempty signatures.

Proof. Positivity is preserved by successful block deletion. A retained port is the result of a failed singleton trial; its recorded zero argument contains the final set with that port removed. Lemma 4.1 applies unchanged.

Fix the final positive set T . A failed block D must meet T : otherwise $T \subseteq U \setminus D$ at the failed trial, contradicting the zero residual and the positive mass at T . At any fixed depth of the balanced tree the node blocks are disjoint, so at most d failed nodes occur there, and at most 2^j occur at depth j . Only failed internal nodes create children. Hence the total visited nodes are at most

$$1 + 2 \sum_{j=0}^{\lceil \log_2 r \rceil - 1} \min(2^j, d).$$

Splitting this sum at depth about $\log_2 d$ gives the stated bound. The entire tree has $2r - 1$ nodes, which gives the universal bound without asymptotic notation. $\square$

This strategy is particularly attractive for many small-cardinality signatures. It need not beat the linear strategy when most ports are retained. The implementation keeps **linear** as its conservative default and makes **split** explicit. Neither choice changes correctness or the polynomial local bound.

4.3 What the query bound does not say

The general coverage reconstruction bound is established literature [2]. A simple counting argument also explains why a dependence on both support size and the number of ports should be expected in a pure value-oracle model. With unit weights and s distinct unknown support sets, there are $\binom{2^r}{s}$ possibilities. Each subset-sum answer lies in $\{0, \dots, s\}$. Thus a decision tree needs at least

$$\frac{\log_2 \binom{2^r}{s}}{\log_2(s+1)}$$

queries in the worst case. For example, when $2 \leq s \leq 2^{r/2}$, this is $\Omega(sr/\log(s+1))$. This is consistent with the coverage lower bound in [2, Theorem 2.4].

That argument hides the input behind an oracle. It is not a lower bound for algorithms that inspect the explicit interval description, and certainly not a lower bound for unknot recognition. Direct weighted orbit methods can exploit more structure than a value-oracle interface exposes.

5 Certificates without the exploratory search trace

A producer's successful search steps are not all needed to certify its final histogram. For each emitted atom T_j of weight w_j , retain a proof of the relevant raw value $G(T_j)$ and, for each $i \in T_j$, one zero-witness set $Z_{j,i}$ satisfying

$$T_j \setminus \{i\} \subseteq Z_{j,i}, \quad i \notin Z_{j,i}. \quad (5)$$

The checker verifies

$$G(Z_{j,i}) = \sum_{\ell < j, T_\ell \subseteq Z_{j,i}} w_\ell, \quad (6)$$

$$G(T_j) - \sum_{\ell < j, T_\ell \subseteq T_j} w_\ell = w_j > 0. \quad (7)$$

It also checks distinct supports and $\sum_j w_j = C$.

The witness set need not equal $T_j \setminus \{i\}$. An earlier failed trial can be a larger set, allowing the producer to reuse a proof it already obtained. This is why the certificate does not need to rerun the search with final supports after the fact.

Theorem 5.1 (Soundness and completeness of sparse replay). *Suppose the supplied orbit proofs are valid for the baseline and the coned unions used in equations (6)–(7). Then the independent checker accepts only the exact complete positive incidence histogram. Every completed run of either recovery strategy produces an accepted certificate.*

At most

$$1 + \sum_{T \in \text{supp } h} (1 + |T|) \quad (8)$$

ordinary orbit-count proofs are sufficient, including the baseline; empty complement unions and repeated canonical unions reduce this count. The checker does not call an orbit-count producer or inspect all subsets of the port set.

Proof. Valid native proofs and Lemma 3.1 establish the exact raw values used by the checker. The true underlying histogram is nonnegative because its coefficients count equivalence classes.

Induct on the entries. Assume earlier entries are correct. For any proper subset $S \subsetneq T_j$, choose $i \in T_j \setminus S$. By (5), $S \subseteq Z_{j,i}$. Equation (6) states that the true remaining nonnegative mass in $Z_{j,i}$ is zero. Therefore there is no remaining mass at S . It follows that the residual at T_j is precisely the true remaining coefficient there, and (7) identifies it as w_j . This preserves the induction hypothesis. Total mass then excludes any missing positive coefficient.

For completeness, each retained singleton in either producer has exactly such a zero witness; later deletions preserve (5). The producer's weight calculation and termination test give the other equations. The empty atom requires only its weight value and no zero witness.

There is one baseline proof, at most one coned proof for each atom weight, and at most one for each retained-port witness. Every proof can be shared by all requests with its exact normalized union. No successful exploratory deletion needs to be retained merely because it was explored. $\square$

5.1 Why positivity is an essential hypothesis

The certificate is not a general tester that determines whether an arbitrary function is a nonnegative coverage function. It proves an output about a concrete equivalence relation. Nonnegativity follows from that input’s semantics, and the native proofs authenticate the needed evaluations. With a signed measure, zero subset mass could hide nonzero coefficients that cancel, invalidating the proof. Treating signed formal coefficients as component counts would therefore be an unsound generalization.

Proposition 5.2 (An atomwise witness lower bound). *In a pure exact subset-sum model with known total mass $C > 0$, a certificate distinguishing the one-atom measure $C \delta_T$ from every alternative $C \delta_{T \setminus \{i\}}$, for $i \in T$, needs at least $|T|$ zero-valued subset-sum queries. One zero query cannot distinguish two of those alternatives.*

Proof. A query containing T has value C for the original measure and for every alternative, so it cannot distinguish them. A query U that distinguishes the alternative omitting i must contain $T \setminus \{i\}$ but not T . Hence it excludes i and includes every other element of T . The same U cannot also contain $T \setminus \{j\}$ for $j \neq i$. Distinct alternatives require distinct zero queries. $\square$

This justifies the retained-port witnesses within this certificate model. It does not prove optimality among all geometric certificates, which may use structural information rather than subset-sum values.

5.2 Independent implementation and trust boundary

`sparse_incidence_verify.py` imports neither `sparse_incidence.py`, the generic recovery module, nor the native orbit producer. It imports only the native local orbit-trace verifier. It independently normalizes marked unions, constructs their coning pairings, validates masks, and checks the equations above. The verification order does not rely on the producer’s scheduling choices.

A count bundle contains one baseline proof and a list of distinct nonempty canonical unions with their proofs. The verifier rejects duplicate unions, unused union proofs, missing witnesses, incorrect source bindings, altered weights, and incomplete total mass. Integer fields reject booleans despite Python’s equality between `True` and `1`; explicitly encoded hexadecimal integers are accepted. Local proof correctness is inherited from the pinned native checker, whose source is included for audit.

Polynomial verification time is measured in the *supplied serialized certificate and integer bit lengths*. It is not an untrusted-input memory containment guarantee. A service accepting external certificates should impose a serialized-size allowance before parsing and an independent deadline during replay.

6 Per-signature parity profiles without a second support search

Suppose each generating relation has a parity $\epsilon \in \mathbb{F}_2$. A component is *consistent* if every closed relation walk has total parity zero. Its two-sheeted cover has universe $X \times \mathbb{F}_2$ and relations

$$(x, e) \sim (g(x), e + \epsilon_g).$$

It is encoded by two blocks of N points and $2k$ interval pairings. Lift every port to both sheets:

$$\hat{A}_i = A_i \times \mathbb{F}_2.$$

No list of $2N$ vertices is constructed.

Lemma 6.1 (Support is preserved by the parity double). *Every lifted component over a base component projects onto that entire base component and has the same port signature. A consistent component has two lifted components; an inconsistent component has one. Consequently, the lifted positive signature support equals the base positive support.*

Proof. Fix a base component and a point in it. Lift a relation path from that point to any other point. Its endpoint lies in the same lifted component as the chosen lift of the starting point. Thus the projection of that lifted component contains every base point. Since each lifted port contains both sheets, port membership of the component is preserved in both directions.

If every closed walk has parity zero, the parity transported from a chosen root is path-independent. The two choices of root sheet produce disjoint connected components. If an odd closed walk exists, it connects the two lifts of its root; path lifting then connects both sheets everywhere. These are the only two possibilities. $\square$

Let $o(T)$ and $u(T)$ be the numbers of consistent and inconsistent base components of signature T . With $\hat{h}$ denoting the lifted histogram,

$$h(T) = o(T) + u(T), \quad \hat{h}(T) = 2o(T) + u(T). \quad (9)$$

Hence

$$o(T) = \hat{h}(T) - h(T), \quad u(T) = 2h(T) - \hat{h}(T). \quad (10)$$

Theorem 6.2 (Known-support parity reconstruction). *After one base recovery with ordered supports $T_1, \dots, T_s$, the complete per-signature parity profile can be computed with at most $s + 1$ additional orbit counts: one lifted baseline and at most one coned lifted query for each support. It has a source-bound independent certificate and polynomial total bit complexity for binary interval input.*

Proof. By Lemma 6.1, there are no additional lifted support sets to discover. By Theorem 4.2, the base output order places every positive proper subset before its superset. Therefore triangular subtraction gives

$$\hat{h}(T_j) = \hat{G}(T_j) - \sum_{\ell < j, T_\ell \subsetneq T_j} \hat{h}(T_\ell). \quad (11)$$

There is only one requested lifted subset sum per support, and Lemma 3.1 implements it by at most one orbit count. The baseline supplies the total and any empty-complement queries. Equations (10) then determine the requested profile.

The signed verifier first checks the base certificate, constructs the parity double independently from the supplied signed pairings, and verifies the necessary lifted count proofs. It checks (11),

total lifted mass, and $h(T) \leq \hat{h}(T) \leq 2h(T)$ before checking (10). Soundness follows from the cover lemma rather than an unproved assumption that the lifted support is small. The double has polynomially larger encoded input, so Theorem 1.1 and the displayed query bound give the complexity claim. $\square$

For actual surface gluings, consistency means orientability only when the supplied parity is the surface orientation character. A reversal of the *ordering of points* in an interval pairing is not by itself that character. The API calls its output “consistent” and “inconsistent” and does not infer a geometric interpretation from the reversal flag.

Corollary 6.3 (Support reuse for other finite covers). *For a connected-base-component covering construction in which each lifted component projects surjectively and all ports are lifted by full inverse image, the lifted histogram has exactly the base signature support. Its multiplicities can be recovered by (11) with one subset-sum value per known support.*

A single such histogram generally does not determine the distribution of all possible lifted-component multiplicities in a higher-degree cover. The special two-sheeted case has exactly two possibilities and the invertible system (9); that additional conclusion must not be silently generalized.

7 Bit complexity and the precise quasi-polynomial consequence

7.1 Input size and oracle-sensitive accounting

Let $\mathcal{L}$ denote the bit length of the explicit interval input, including port delimiters. Up to harmless encoding factors,

$$\mathcal{L} = O(r + (k + M + 1)B).$$

An emitted mask has r bits and every true count has $O(B)$ bits. Write $P(k, B)$ for a polynomial bound on the native unweighted count kernel, including proof production when that option is enabled. The inherited classical orbit bound supplies the existence of such a polynomial; this report does not assert a newly optimized exponent for the native scheduler.

Let q be the number of requested raw subset values after mask caching and let $\mathcal{Q}$ be the set of distinct nonempty canonical complement unions actually queried. For $A \in \mathcal{Q}$, let m_A be its number of interval constituents. The count-kernel portion is bounded by

$$P(k, B) + \sum_{A \in \mathcal{Q}} P(k + 2m_A, B). \quad (12)$$

This is often more informative than $(q+1)P(k+2M, B)$. Adjacent marked intervals can collapse into few constituents; separated singletons may not. The experiments in Section 8 show that identical r, s, k, M, B orders of magnitude do not imply similar actual query costs.

The simple implementation scans previously recovered entries when evaluating a residual. There are $O(sr)$ trials in the linear strategy, hence $O(s^2r)$ support-containment tests and bounded-integer additions in this straightforward realization. Canonical-union construction and comparison also have polynomial cost. Python dictionaries are used for exact caches. Expected dictionary behavior is useful for timings, but even a pessimistic linear scan of all previously stored keys adds only polynomial overhead. No probabilistic hashing assumption is needed for the polynomial-time classification.

Thus an unconditional coarse form of the recovery bound is

$$T \leq P(k, B) + \sum_{A \in Q} P(k + 2m_A, B) + \text{poly}(\mathcal{L}, s). \quad (13)$$

The memory bound is also polynomial in $\mathcal{L}, s$, including the cached traces when recording is enabled. The final certificate need not retain all of those exploratory traces: Theorem 5.1 keeps only the ones required by its equations. Certificate verification is polynomial in that retained representation.

7.2 Closing the local complexity argument

By Lemma 2.2, s is polynomially bounded in $\mathcal{L}$. Since $q = O(sr + 1)$ and $m_A \leq M$, equation (13) is polynomial in $\mathcal{L}$. The same substitution makes the certificate count, metadata length, and replay cost polynomial. This proves Theorem 1.1.

The conclusion is stronger than a condition of the form “polynomial time when $r = O(\log n)$ ”: for this sparse local query, r may be any polynomially bounded part of the encoded input. It is weaker than a theorem about a full unknot algorithm, because the encoded inputs generated by a global search may not remain polynomially bounded.

For the *unchanged dense output contract*, no such elimination of 2^r is possible. Even an all-zero-port instance forces a literal dense output of 2^r slots. The result requires changing the interface, not merely speeding up its internal Möbius transform.

7.3 A conditional transfer statement for a hierarchy

Proposition 7.1 (Incidence workload along a supplied hierarchy). *Consider a computation associated with an n -crossing source that makes at most $Q(n)$ incidence or signed-incidence calls, each on an explicit interval description of length at most $L(n)$. Replacing dense incidence by the sparse routines bounds their total contribution by*

$$Q(n) \text{ poly}(L(n)).$$

If both $Q(n)$ and $L(n)$ are at most $\exp(O((\log n)^2))$, this contribution is quasi-polynomial in n . No separate logarithmic restriction on the number of ports is needed.

Proof. Use one fixed polynomial bound from Theorems 1.1 and 6.2 for each call and sum it over the calls. Raising an $\exp(O((\log n)^2))$ bound to a fixed power and multiplying two such bounds preserves that form. $\square$

The proposition does not assert its hypotheses for the existing recognizer. It also does not cover other work in the computation. A full result still needs bounded, source-correct construction of the current hierarchy state; compressed cutting and reattachment with the relevant maps; bounds on search, restarts, and depth; and a complete terminal recognition argument. A signature histogram alone cannot substitute for ordered attachment data. This contribution removes a dense-incidence obstruction in that accounting, not all of these obligations.

8 Validation and controlled performance measurements

All mathematical outputs use exact integer arithmetic. Floating-point numbers appear only as elapsed-time measurements. The native files in the package are exact byte copies of the inspected

repository blobs; the loader checks their Git object hashes before importing them. The experiment uses CPython 3.13.5 on Linux x86-64. The full interpreter and platform strings, source pins, seeds, every timing sample, and structural counts are in the retained JSON files.

8.1 Correctness checks actually performed

The delivered test suite has 21 test methods and passes in the recorded final run. Its loops include 500 random small interval systems, each compared with literal graph components under both recovery strategies; 1,000 arbitrary nonnegative histograms under both generic strategies; and 300 random signed systems compared with an independent graph parity traversal. Every interval and signed result in these loops is also checked through the independent sparse verifier.

A separate exhaustive audit realizes all 256 zero-one histograms on three ports as static interval systems. Both sparse strategies agree with the native dense implementation and with the known histogram, giving 512 additional comparisons and 512 independent sparse replays. The eight main benchmark output digests are separately checked against closed-form descriptions of their fixtures, not only against a second invocation of the same orbit kernel.

Other checks cover empty universes, unmarked components, overlapping and coincident ports, reflection fixed points, source mutations, coefficient mutations, missing or invalid witnesses, repeated supports, incomplete total mass, shared resource limits, and interruption. Replay remains successful with producer functions disabled. A certificate involving $N = 2^{16000}$ is transported through JSON using exact hexadecimal integers and verified without changing Python’s global decimal-conversion setting. These are tests and audits, not a formal verification of the Python implementation.

The repository reports a larger maintained suite, but that full suite was *not* run in this environment. We claim only the shipped local tests and audits over the pinned native modules. No end-to-end diagram-recognition timings were collected.

8.2 Main benchmark design

The main comparison has eight fixtures and five shuffled arms: native dense incidence; sparse linear deletion; sparse block deletion; an identical block-deletion control; and block deletion with certificate production plus independent replay. Each arm has one warm-up batch and five measured batches of two calls. This gives 400 timed calls and 80 warm-up calls. Fixture construction is outside the timed region; API validation, normalization, query processing, and requested proof operations are inside. JSON serialization is outside timing.

The disjoint, coincident, and nested large-block fixtures use $L = 2^{256}$. The paired-block fixture has two supplied long pairings, one order-preserving and one order-reversing. Its nine signatures have known multiplicities. The all-signatures fixture has 64 static points and all 64 possible signatures on six ports. That input deliberately defeats sparsity.

Workload	s	Dense	Linear	Split	Split + replay
Disjoint, $r = 6$	7	2.094	0.606	0.370	0.670
Disjoint, $r = 8$	9	13.153	1.003	0.565	1.039
Disjoint, $r = 10$	11	73.592	1.666	0.717	1.352
Coincident, $r = 4$	2	0.046	0.029	0.036	0.084
Coincident, $r = 10$	2	3.026	0.069	0.076	0.175
Nested, $r = 10$	11	2.979	0.347	0.455	0.855
Paired blocks, $r = 8$	9	24.968	2.181	1.335	2.114
All signatures, $r = 6$	64	16.122	17.324	18.003	26.767

Table 1: Main median wall times in milliseconds. Dense, linear, and split are uncertified queries. The last column includes sparse certificate production and independent replay; it is not a like-for-like timing of two certification backends.

Workload	Dense queries	Split queries	Dense bytes	Sparse bytes
Disjoint, $r = 6$	64	14	96,337	15,099
Disjoint, $r = 8$	256	21	502,773	20,805
Disjoint, $r = 10$	1024	26	2,498,706	26,538
Coincident, $r = 4$	2	2	2,338	2,574
Coincident, $r = 10$	2	2	7,342	3,606
Nested, $r = 10$	11	11	12,896	10,725
Paired blocks, $r = 8$	256	21	730,590	35,055
All signatures, $r = 6$	64	64	42,130	49,505

Table 2: Physical orbit-count calls and compact-JSON certificate sizes. Both dense and sparse certificates were independently verified. Certificate serialization was not timed.

On the disjoint 10-port instance, the count-query reduction is exactly 1,024 to 26. The native dense certificate has 2,498,706 bytes, whereas the sparse certificate has 26,538 bytes. These are measurements of the same mathematical histogram using the same orbit kernel, not comparisons against a literal expansion of 2^{256} points. The main timing ratio is about 10^2 in favor of block deletion.

Coincident and nested cases are instructive for a different reason. The native cache already reduces the physical counts to two and eleven, respectively. Sparse recovery still avoids dense mask traversal and inversion. The gain there is not a claim that additional orbit-query sharing was newly discovered.

The six-port all-signatures control is unfavorable: sparse residual bookkeeping pays overhead without saving an orbit query. The main block-deletion time is about 18.0 ms versus 16.1 ms for dense, and its certificate is larger. The default linear policy is better than block deletion on the nested and coincident fixtures. These results argue against silently replacing all small dense calls by one universally preferred sparse policy.

8.3 Longer-batch follow-up and timing uncertainty

The initial identical-arm paired ratios ranged roughly from 0.877 to 1.049, with the largest deviation on the 10-port disjoint instance. We retained those data and ran a separate seven-round, ten-repetition follow-up on that fixture, the paired-block fixture, and the unfavorable full-support control.

The follow-up contains 1,050 measured calls and 150 warm-up calls.

Workload	Dense (ms)	Split (ms)	Paired gain	Split/control
Disjoint, $r = 10$	78.549	0.732	106.76	0.975
Paired blocks, $r = 8$	28.110	1.228	22.85	0.946
All signatures, $r = 6$	17.308	20.011	0.87	1.000

Table 3: Longer-batch follow-up. “Paired gain” is the median of same-round dense/split ratios, not the ratio of the displayed medians. Split/control compares identical implementations.

The follow-up preserves the order-of-magnitude positive conclusions and the unfavorable full-support conclusion. It does not justify precise universal speed factors. The sample families are selected, system timing noise is visible, and no formal statistical significance analysis is claimed. The two rounds of experiments remain separate in the package rather than being pooled into a more favorable number.

8.4 Large port sets and a geometric negative control

For $N = 2^{500}$, take r adjacent singleton ports $[i, i + 1)$. There are $r + 1$ positive signatures: one for each singleton and the unmarked remainder. Table 4 reports three complete certified calls per size, including replay. A dense call is not attempted at these sizes; no speedup ratio is assigned to an unrun baseline.

r	s	Count queries	Query + replay (ms)	Certificate bytes
32	33	113	6.190	48,881
128	129	577	65.049	202,530
256	257	1281	215.383	422,028

Table 4: Adjacent singleton ports on a binary 2^{500} -point static universe. A literal dense answer has 2^r slots, but the sparse answer has $r + 1$ entries.

Move the singleton ports to $[2i, 2i + 1)$ and their complement unions generally have many more constituents. The 32-port certified query and replay completed in about 0.16 seconds, compared with about 0.006 seconds for the adjacent version in the main large-case experiment. The 128- and 256-port gapped controls exceeded a separate five-second cooperative driver allowance; no histogram or relative speed ratio is published for those unfinished calls. The allowance covered both production and replay.

An earlier exploratory benchmark process was stopped at 200 seconds during its gapped large-case phase. Its completed small-case log is retained as `results/benchmark_aborted.log`; no complete large-case timings survived that run. This failure prompted the explicit censored controls, not a claim that a timeout proves an asymptotic lower bound.

The negative control explains why the oracle-sensitive bound (12) matters. Polynomiality does not mean that arbitrary 256-port instances are inexpensive, and equal support size does not remove the cost of complicated coned unions.

8.5 Parity support reuse

The signed experiment compares two complete workloads over the same native kernel: recovering base and lifted supports independently, versus recovering the base support once and using triangular lifted queries. Each has one warm-up round and five shuffled measured rounds. The fixtures contain long reflected blocks and parity-one identity relations on selected blocks, so both consistent and inconsistent components occur. Certificates were additionally produced and independently replayed outside the timings in Table 5.

r	s	Cover: two searches	Cover: reuse	Two searches (ms)	Reuse (ms)
8	9	21	10	397.635	276.153
16	17	49	18	1813.532	1110.707

Table 5: Signed-profile query times without certificates. The cover-query columns isolate the saved second support search. Both arms include the base computation.

For eight ports, lifted count calls fall from 21 to 10; for sixteen, from 49 to 18. The paired whole-profile gains are about 1.45 and 1.61. These are local signed-incidence improvements. They are not timings for recognizing a knot or for computing a whole surface hierarchy.

9 Software contract and integration

9.1 Additive modules rather than a hidden dispatch change

The integration directory contains three new modules: `_sparse_coverage.py`, `sparse_incidence.py`, and `sparse_incidence_verify.py`. They are designed to be copied into the maintained `fast/fastunknot/` package. An additive patch and a native-style test file are also supplied. No existing recognizer file is overwritten by the patch.

The unsigned entry point is

```
analyze_sparse_port_incidence(
    size, pairings, ports, strategy="linear",
    max_ports=None, max_signatures=None,
    max_cycles=None, max_queries=None,
    record_certificate=False, check=None)
```

All arguments following `ports` are keyword-only in the actual Python signature. A completed result contains `histogram` as a list of `[mask, positive_count]` rows. It is not the positional dense list returned by the existing interface. Consumers must explicitly adopt the new output type. Converting it into a dense list merely reintroduces the exponential output cost.

The signed entry point has the analogous options and returns `signed_histogram` rows of the form `[mask, consistent_count, inconsistent_count]`. A separate ordinary histogram records their sums. Signatures absent from the sparse list have zero multiplicity; the output order is a valid peeling order, not a promise of numerical sorting of masks.

9.2 Resource accounting

Every physical orbit count shares one total `max_cycles` allowance. The signed API shares that allowance between the base recovery and the parity double. `max_queries` limits physical orbit invocations; exact cache hits consume no additional count query. `max_signatures` limits emitted positive atoms. These are explicit controls, not structural assumptions in the unlimited theorem.

Exhaustion of an internal allowance produces `INCONCLUSIVE`, a reason, and work statistics, with no histogram or certificate. An external cooperative deadline propagates its interruption rather than becoming a negative mathematical verdict. As with the native kernel, a cycle allowance is not a calibrated bound on elapsed seconds or bit operations. Canonicalization, residual arithmetic, and replay also poll the callback.

9.3 Minimal reproduction

The archive can be tested offline using its immutable native snapshots:

```
python tools/bootstrap.py
python -m unittest discover -s tests -v
python tools/audit.py
python tools/benchmark.py --repeats 2 \
    --output results/benchmark_rerun.json
```

The audit checks the retained main fixture digests and writes audit receipts and example certificates. The benchmark command writes new timing data without overwriting the retained primary measurements. Longer batches are available in `tools/followup.py`. Table regeneration uses the retained results by default:

```
python tools/make_tables.py
cd paper
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The integration patch is a proposal for the repository owner to review and apply. It was checked in a clean local Git work tree and its native-style tests were executed against the included exact kernel snapshots. It has not been committed to or run through the complete current ProveIt repository. `INTEGRATION.md` gives the dependency pins, installation boundary, and recommended regression checks.

10 Further research questions and concrete proof targets

10.1 A direct, certified weighted orbit implementation

The most immediate alternative is an implementation of weighted AHT with independently replayable weight transfer. It may avoid the many coned counts in equation (12). The proof obligation is not just total orbit count preservation: every transfer must preserve each orbit's vector sum and multiplicity. A useful comparison should hold the native geometry and input validation fixed, report certificate size, and include separated-port controls. The present reduction supplies an independent oracle-based reference for that implementation.

10.2 Explicit support bounds in breakpoint parameters

Lemma 2.2 deliberately inherits an unspecified fixed polynomial. A sharper quantitative theorem could track how many constant-weight intervals are created and removed during certified weighted reductions. The target is a proved explicit bound in M , k , and B , preferably separating the effect of the vector dimension r . Such a bound could guide memory budgets and identify instances where sparse recovery should be selected before running it. The coarse existence theorem alone is not a good predictor of practical cost.

10.3 Better coning plans for separated marks

The gapped-singleton failure is a concrete optimization target. The current star of singleton joins matches the maintained coning rule and simplifies source binding. Other connection trees, partial periodic joins, or structured auxiliary relations may produce the same quotient with a cheaper AHT schedule. A valid improvement must prove the same orbit-merger identity, certify its chosen relations, and compare against the unchanged star on both favorable and adverse patterns. Merely decreasing the number of joins cannot help asymptotically when all marked constituents must be connected; their geometry and scheduling also matter.

10.4 Adaptive recovery and certified residual data structures

The simple $O(s^2r)$ residual bookkeeping is visible on dense-support controls. Inclusion-query data structures, prefix refinements, or an adaptive choice between linear and block deletion may reduce it. The challenge is to preserve exact residuals after every emitted coefficient and keep the verifier simpler than the optimizer. A useful theorem would give an instance-sensitive bound that includes support cardinalities, containment structure, and the sizes of the coned unions actually queried, rather than selecting a policy from r alone.

10.5 Stable port families under local updates

A hierarchy changes both interval relations and ports. Exact reuse is straightforward when the entire normalized source query is unchanged; it is not justified merely because two old counts match. A stronger dynamic interface would certify how an old component signature transforms under a specified update. The desired object is a checked transition map or refinement certificate, not an unproved cache heuristic. Determining which hierarchy moves admit inexpensive such updates is a natural next step.

10.6 From incidence sets to ordered attachment data

Histograms forget point multiplicity within a port, cyclic order, slope, and which specific intervals are glued next. Different attachment structures can have identical histograms. The key topological question is whether a canonical compressed representation of those additional data can be maintained with a polynomial local update bound. Incidence and parity certificates could be components of that representation, but cannot be substituted for it. A first concrete target is a verified normal-arc adapter carrying source and attachment provenance through a single compressed cut.

10.7 Finite covers and topology by signature

The support-reuse corollary suggests computing several lifted statistics on the same support. For higher-degree covers, determine which collections of lift counts distinguish the desired monodromy or component types and how many queries are necessary. For surfaces, a complementary task is to combine verified orientation data with Euler characteristic and boundary information *per signature*, without assuming that aggregate counts determine every component type. The two-sheeted formulas in this paper provide a small exactly understood case.

10.8 The global size and completeness theorem

The decisive remaining research target is a bound relating the entire evolving compressed hierarchy to the original crossing number. Proposition 7.1 makes one part of the needed accounting explicit: the number and encoded sizes of incidence calls. The full theorem must also bound all other operations and prove that the search reaches a valid terminal certificate. Establishing those hypotheses, or finding a different complete recognition route with comparable bounds, is a substantive mathematical problem that is not discharged by the local experiments here.

11 Conclusion

The dense incidence table is not intrinsically necessary for binary interval relations. Weighted orbit theory bounds the number of positive signatures polynomially, and nonnegative residual recovery makes them accessible through the repository’s existing unweighted kernel. Compact minimality witnesses allow independent replay without preserving every exploratory query. The parity double adds a second useful statistic without a second support search.

The result is a concrete, tested local asymptotic improvement and a more favorable interface for future compressed-topology work. It comes with substantial gains on some native-kernel workloads, measured regressions on full-support input, and explicit censored separated-port controls. None of these observations supplies a general quasi-polynomial unknot recognizer. The next progress should connect certified local state to source-correct geometric transitions and bounded global search, rather than treating a sparse histogram as an attachment map.

A A completely explicit 17-point example

Take the static relation on $X = \{0, \dots, 16\}$ and three ports

$$A_1 = [2, 10), \quad A_2 = [5, 17), \quad A_3 = [10, 17).$$

The only positive coefficients are

$$h(\emptyset) = 2, \quad h(\{1\}) = 3, \quad h(\{1, 2\}) = 5, \quad h(\{2, 3\}) = 7.$$

The total is 17. With linear deletion order 1, 2, 3, the implementation emits the empty set, then $\{2, 3\}$, then $\{1\}$, then $\{1, 2\}$. This is inclusion-compatible but not sorted by cardinality.

After removing the empty weight, deleting port 1 leaves residual $G(\{2, 3\}) - 2 = 9 - 2 = 7$. Further singleton deletions give zero, so the first nonempty atom is $\{2, 3\}$ with weight seven and

witnesses $\{3\}$ and $\{2\}$. After that atom is removed, $\{2, 3\}$ itself is a zero witness for retaining port 1 in the next pass. Ports 2 and 3 can be deleted, yielding $\{1\}$ with weight three. The final pass yields $\{1, 2\}$ with weight five.

The exact certificate is saved in `examples/basic_sparse_certificate.json`. Its mask encoding is zero-based, so the positive rows are

```
[[0, 2], [6, 7], [1, 3], [3, 5]]
```

The input ports, source relation, all necessary native count proofs, atom weights, and zero witnesses are included. The proof checker reconstructs coning from the supplied input rather than trusting the producer’s interval descriptions alone.

B Certificate transport and source provenance

The unsigned certificate has a version and kind tag, universe size, normalized ports, total orbit count, ordered entries, and a count bundle. Each entry contains a mask, positive weight, and one pair `[bit, zero_mask]` for every set bit. The count bundle contains the baseline native proof and distinct canonical nonempty union proofs. A signed certificate contains an unsigned base certificate, a lifted count bundle, lifted weights in the base order, and the claimed parity split.

The following native Git blob identifiers are checked by `tools/bootstrap.py`. They identify file bytes, not a claim that the whole repository was atomically checked out at one commit.

Native file	Git blob SHA-1
<code>interval_orbits.py</code>	<code>e86050aadbcef6f17fdddf9cbd1abc0e7ca18e8</code>
<code>interval_orbit_verify.py</code>	<code>0ccb56a0e8b5f1255384121d7417441314727720</code>
<code>interval_incidence.py</code>	<code>3625772c7c20119df64d79a14f644a03e13f7f2f</code>
<code>integer_codec.py</code>	<code>b88eafacb680e1312df0ecc93170a9f6f53eb639</code>

Large integer transport uses an explicit hexadecimal string only when needed; no decimal-limit setting is changed. Masks and counts are decoded only in fields where integers are part of the schema. Canonical union records are bound to exact source pairings through their native proofs. Cryptographic hashes document provenance and reproducibility, not mathematical correctness by themselves.

C Reproducibility and claim ledger

The archive separates `integration/` from `snapshots/`, so new code is not confused with the maintained baseline. `results/benchmark.json` contains the primary timing samples; `benchmark_followup.json` retains the longer-batch comparison; `audit.json` records exact additional checks and censored controls. Test logs, example certificates, an additive patch, source pins, and a SHA-256 manifest are included.

The strongest unconditional mathematical statement is polynomial-time sparse incidence on explicitly encoded interval systems. The strongest signed statement is exact per-signature consistency with support reuse. The practical gain is a measured improvement on selected local workloads against the exact maintained dense code. The global quasi-polynomial conclusion is only the conditional incidence-workload statement in Proposition 7.1; no complete knot algorithm meeting

those hypotheses is supplied. This separation should be retained if the contribution is integrated into the maintained synthesis article.

References

- [1] I. Agol, J. Hass, and W. Thurston, *The computational complexity of knot genus and spanning area*, Transactions of the American Mathematical Society **358** (2006), 3821–3850. Preprint arXiv:math/0205057, version 2. Theorem 12 is the unweighted orbit bound; the compressed weighted representation preceding Theorem 16 and Theorem 16 are the inputs used here.
- [2] D. Chakrabarty and Z. Huang, *Testing Coverage Functions*, arXiv:1205.1587, 2012. Section 2 gives $O(rs)$ coverage reconstruction and the associated query lower bound. The present article does not claim those generic bounds as new.
- [3] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, July 2026. See in particular Section 9 for the running-time discussion.
- [4] Mathematical Institute, University of Oxford, *Marc Lackenby announces a new unknot recognition algorithm that runs in quasi-polynomial time*, 2021. <https://www.maths.ox.ac.uk/node/38304>. Used as historical context, not as an executable specification for this package.
- [5] V. Reshetnikov, *ProveIt, Topology/UnknotRecognition*, inspected 8 October 2026. GitHub repository and UnknotRecognition subtree. Maintained dense incidence and native count/verifier sources are pinned in Appendix B and included in `snapshots/`. The reviewed `synthesis/orbit_certificates.tex` has Git blob `a83b3ab44d95b2e8c35752266d9db626ca126225`. This reference attributes the existing coning identity, canonical-union cache, local AHT replay, and the documented scope of incidence data.